Unit 3: Constructor and Destructor

Prof. Mahipal Khoja
Assistant Professor
Artificial Intelligence and Data Science Department

Content

- Constructors in C++
- Types of Constructors in C++
  - Default Constructor
  - Parameterized Constructor
  - Copy Constructor
  - Move Constructor
- Characteristics of Constructors
- Destructor

Constructors in C++

In C++, **constructors** are special methods that are automatically called whenever an object of a class is created. A constructor is different from normal functions in following ways:

- A constructor has same name as the class itself
- Don't have return type
- Automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).

Constructors in C++

```
#include <iostream>
using namespace std;
class A {
    public:
    // Constructor of the class without any parameters
    A() {
        cout  << "Constructor called" << endl;
    }
};

int main() {
    A obj1;
    return 0;
}
```

Output:
Constructor called

Type of Constructor in C++

Constructors can be classified based on the situations they are being used in.

There are 4 types of constructors in C++:

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Move Constructor

Default Constructor

A default constructor is automatically generated by the compiler if the programmer does not define one. This constructor doesn't take any argument as it is parameter less and initializes object members using default values. It is also called a zero-argument constructor. However, the default constructor is not generated by the compiler if the programmer has explicitly defined a constructor.

Default Constructor

```
#include <iostream>
using namespace std;
// Class with no explicitly defined constructors
class A {
    public:
};
int main() {
    // Creating object
    A a;
    return 0;
}
```

In the above program the class A does not contains any explicitly defined constructor. Hence, The object of the class A is created without any parameters, As the class will use the default constructor generated by the compiler.

Parameterized Constructor

Parameterized constructor allow us to pass arguments to constructors. Typically, these arguments help initialize an object's members. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object's members.

Parameterized Constructor

```
#include <iostream>
using namespace std;
class A {
    public:
    int val;
    // Parameterized Constructor
    A(int x) {
        val = x;
    }
};
int main() {
    // Creating object with a parameter
    A a(10);
    cout << a.val;
    return 0;
}
```

Output:
10

Parameterized Constructor

In this code, the parameterized constructor is called when we create object a with integer argument 10 . As defined, it initializes the member variable val with the value 10.

Note: If a parameterized constructor is defined, the non-parameterized constructor should also be defined as compiler does not create the default constructor.

Copy Constructor

A copy constructor is a member function that initializes an object using another object of the same class. Copy constructor takes a reference to an object of the same class as an argument.

```
#include <iostream>
using namespace std;
class A {
    public:
    int val;
    // Parameterized constructor
    A(int x) {
        val = x;
    }
```

Copy Constructor

```
    // Copy constructor
    A(A& a) {
        val = a.val;
    }
};
int main() {
    A a1(20);
    // Creating another object from a1
    A a2(a1);
    cout << a2.val;
    return 0;
}
```

Output:
20

Copy Constructor

In this example, the copy constructor is used to create a new object a2 as a copy of the object a1. It is called automatically when the object of class A is passed as constructor argument.

Just like the default constructor, the C++ compiler also provides an implicit copy constructor if the explicit copy constructor definition is not present.

Note: Unlike the default constructor where the presence of any type of explicit constructor results in the deletion of the implicit default constructor, the implicit copy constructor will always be created by the compiler if there is no explicit copy constructor or explicit move constructor is present.

Move Constructor

The move constructor is a recent addition to the family of constructors in C++. It is like a copy constructor that constructs the object from the already existing objects. ,but instead of copying the object in the new memory, it makes use of move semantics to transfer the ownership of the already created object to the new object without creating extra copies. It can be seen as stealing the resources from other objects.

The move constructor uses std::move() to transfer ownership of resources and it is called when a temporary object is passed or returned by value.

Move Constructor

```
#include <bits/stdc++.h>
using namespace std;
class MyClass {
    private:
        int b;
    public:
        // Constructor
        MyClass(int &&a) : b(move(a)) {
            cout << "Move constructor called!" << endl;
        }
    void display() {
        cout << b <<endl;
    }
};
```

Move Constructor

```
int main() {
    int a = 4;
    MyClass obj1(move(a));  // Move constructor is called
    obj1.display();
    return 0;
}
```

Explanation: In the above program, MyClass uses a constructor with an **rvalue reference**(int&&) to move a value into its member using std::move. When an object is created with std::move, the move constructor transfers ownership of resources instead of copying them. If you don't define one, the compiler automatically generates a move constructor (just like it does for copy constructors).

Characteristics of Constructor

- The name of the constructor is the same as its class name.
- Constructors are mostly declared as public member of the class though they can be declared as private.
- Constructors do not return values, hence they do not have a return type.
- A constructor gets called automatically when we create the object of the class.
- Multiple constructors can be declared in a single class.
- In case of multiple constructors, the one with matching function signature will becalled.

Destructor

Destructor is an instance member function that is invoked automatically whenever an object is going to be destroyed. Meaning, a destructor is the last function that is going to be called before an object is destroyed.

Destructors are automatically present in every C++ class but we can also redefine them using the following syntax.

```
~className(){
    // Body of destructor
}
```

where, **tilda(~)** is used to create destructor of a **className**.

Destructor

Just like any other member function of the class, we can define the destructor outside the class too:

```
className {
public:
   ~className();
}

class-name :: ~className() {
   // Desctructor Body
}
```

But we still need to at least declare the destructor inside the class.

Destructor

Examples of Destructor

```
#include <iostream>
using namespace std;

class Test {
public:

    // User-Defined Constructor
    Test() {
        cout << "Constructor Called" << endl;
    }
```

Destructor

```
    // User-Defined Destructor
    ~Test() {
        cout << "Destructor Called" << endl;
    }
};
int main() {
    Test t;
    return 0;
}
```

Output:
Constructor Called
Destructor Called

Parul® University

NAAC GRADE A++

https://paruluniversity.ac.in/

PU